# LAPORAN CAPSTONE PROJECT

**Klasifikasi Diagnosis Kanker Payudara**

Capstone I, Capstone II, dan Capstone III

Azzahra Putri Kamilah - 123450013

Malika Azzahra - 123450090

Alliya Ammara Ananta - 123450075

## BAB I. PENDAHULUAN

### 1.1 Latar Belakang

Kanker payudara merupakan salah satu kondisi yang memerlukan proses pemeriksaan dan diagnosis secara cermat. Dalam kegiatan capstone ini, data karakteristik sel dari dataset Breast Cancer Wisconsin (Diagnostic) digunakan untuk membangun proses klasifikasi yang dapat membedakan diagnosis jinak (benign) dan ganas (malignant).

Pengerjaan dibagi menjadi tiga tahap. Capstone I berfokus pada persiapan dan pemahaman data, Capstone II berfokus pada pembangunan serta evaluasi model klasifikasi, sedangkan Capstone III berfokus pada penerapan model ke dalam dashboard berbasis Streamlit.

### 1.2 Tujuan

1. Memahami karakteristik dataset dan melakukan pembersihan serta eksplorasi data.
2. Memilih fitur yang relevan dan melakukan reduksi dimensi menggunakan PCA.
3. Membangun dan membandingkan beberapa model klasifikasi.
4. Melakukan hyperparameter tuning dan mengevaluasi model menggunakan metrik klasifikasi.
5. Menerapkan model terpilih ke dalam aplikasi prediksi sederhana berbasis Streamlit.

### 1.3 Problem Statement

Kanker payudara merupakan salah satu penyakit yang membutuhkan deteksi dan penanganan sejak dini. Permasalahan yang ingin diselesaikan dalam proyek ini adalah bagaimana mengolah dan menganalisis data pasien kanker payudara untuk mengetahui karakteristik yang membedakan pasien dengan kondisi tumor jinak (benign) dan ganas (malignant). Melalui proses Exploratory Data Analysis (EDA), data preprocessing, dan feature selection, diharapkan dapat ditemukan fitur-fitur yang memiliki hubungan penting dengan kondisi diagnosis pasien sehingga menghasilkan dataset yang lebih bersih dan siap digunakan untuk proses pemodelan machine learning.

### 1.4 About Datasets

Dataset yang digunakan dalam proyek ini adalah Breast Cancer Wisconsin (Diagnostic) Dataset, yaitu dataset yang berisi hasil pengukuran karakteristik sel jaringan payudara untuk membantu mengklasifikasikan tumor jinak dan ganas.

- Sumber dataset: Breast Cancer Wisconsin (Diagnostic) – UCI Machine Learning Repository.
- Link referensi: https://archive.ics.uci.edu/dataset/17/breast+cancer+wisconsin+diagnostic
- Jumlah data awal: 569 baris.
- Jumlah kolom awal: 33 kolom.
- Jumlah data setelah preprocessing dan penanganan outlier: 398 baris dan 31 kolom.

Dataset terdiri atas kolom ID, diagnosis, dan 30 fitur numerik yang dikelompokkan menjadi tiga jenis pengukuran, yaitu:

1. Mean: Rata-rata karakteristik sel, seperti radius, texture, perimeter, area, smoothness, compactness, concavity, concave points, symmetry, dan fractal dimension.
2. SE (Standard Error): Nilai standar error dari masing-masing karakteristik sel.
3. Worst: Nilai terbesar atau nilai terburuk dari masing-masing karakteristik sel.

Kolom ID merupakan identitas data pasien dan tidak digunakan sebagai fitur pemodelan. Kolom diagnosis merupakan variabel target yang menunjukkan kondisi tumor:

- 0 = Negatif (Benign/Jinak).
- 1 = Positif (Malignant/Ganas).

Setelah proses preprocessing, dilakukan feature selection sehingga diperoleh sembilan fitur utama yang digunakan untuk analisis selanjutnya.

**1.5 Metrics**

Beberapa metrik yang diamati dalam Capstone 1 meliputi:

1. Jumlah data: Jumlah baris dan kolom sebelum dan sesudah proses preprocessing.
2. Distribusi target: Proporsi pasien dengan tumor jinak (Benign) dan tumor ganas (Malignant).
3. Missing values: Jumlah nilai kosong pada setiap kolom untuk mengetahui kelengkapan data.
4. Duplicate data: Jumlah data duplikat yang ditemukan dalam dataset.
5. Outlier: Jumlah data pencilan yang ditemukan dan ditangani menggunakan metode Interquartile Range (IQR).

6. Statistik deskriptif: Nilai rata-rata, median, standar deviasi, minimum, dan maksimum dari fitur numerik.
7. Korelasi fitur: Hubungan antarfitur numerik serta hubungan fitur dengan variabel target.
8. Feature selection: Pemilihan sembilan fitur yang memiliki kontribusi penting untuk analisis dan pemodelan selanjutnya.

Berdasarkan hasil preprocessing, dataset yang awalnya terdiri dari 569 baris dan 33 kolom menjadi 398 baris dan 31 kolom. Distribusi target setelah preprocessing terdiri dari sekitar 75,4% data tumor jinak dan 24,6% data tumor ganas.

## BAB II. CAPSTONE I – DATA PREPARATION

### 2.1 Pengenalan Dataset

Dataset yang digunakan adalah Breast Cancer Wisconsin (Diagnostic). Berdasarkan hasil pembacaan awal, dataset berisi 569 baris dan 33 kolom. Kolom terdiri dari identitas data, diagnosis, 30 variabel numerik yang menggambarkan karakteristik sel, serta satu kolom kosong bernama 'Unnamed: 32'. Kolom ID dan kolom kosong dibuang karena tidak digunakan sebagai variabel prediktor.

Variabel diagnosis diubah menjadi target biner: M (malignant/ganas) diberi nilai 1, sedangkan B (benign/jinak) diberi nilai 0. Dengan demikian, model diarahkan untuk memprediksi apakah sampel termasuk kelas ganas atau jinak.

Gambar 1. Pemeriksaan awal dataset

```
IN [53]:

  d = pd.read_csv(DATA_PATH)
  print(d.shape)
  print(d.columns.tolist())
  print(d.isna().sum())
  d.head()

OUTPUT

  (569, 33)
  ['id', 'diagnosis', 'radius_mean', 'texture_mean', 'perimeter_mean', 'area_mean',
  'smoothness_mean', 'compactness_mean', 'concavity_mean', 'concave points_mean',
  'symmetry_mean', 'fractal_dimension_mean', 'radius_se', 'texture_se', 'perimeter_se',
  'area_se', 'smoothness_se', 'compactness_se', 'concavity_se', 'concave points_se',
  'symmetry_se', 'fractal_dimension_se', 'radius_worst', 'texture_worst',
  'perimeter_worst', 'area_worst', 'smoothness_worst', 'compactness_worst',
  'concavity_worst', 'concave points_worst', 'symmetry_worst', 'fractal_dimension_worst',
  'Unnamed: 32']
  id                         0
  diagnosis                  0
  radius_mean                0
  texture_mean               0
  perimeter_mean             0
  area_mean                  0
  smoothness_mean            0
  compactness_mean           0
  concavity_mean             0
  concave points_mean        0
  symmetry_mean              0
  fractal_dimension_mean     0
  radius_se                  0
  texture_se                 0
  perimeter_se               0
  area_se                    0
  smoothness_se              0
  compactness_se             0
  concavity_se               0
  concave points_se          0
  symmetry_se                0
  fractal_dimension_se       0
  radius_worst               0
  texture_worst              0
```

### 2.2 Data Cleaning dan Data Quality

Tahap pembersihan dilakukan dengan memeriksa tipe data, nilai kosong, nilai yang tidak valid, dan data duplikat. Kolom target diperlakukan sebagai data kategorikal, sedangkan fitur pengukuran sel diperlakukan sebagai data numerik. Hasil pemeriksaan missing value menunjukkan tidak terdapat nilai kosong pada 30 fitur dan target setelah kolom yang tidak digunakan dihapus. Pemeriksaan duplikat juga menghasilkan 0 baris duplikat.

Gambar 2. Pemeriksaan missing value

IN [71]:

```
# Check missing values kembali
print("Hasil missing value:\n", data.isna().sum())
```

OUTPUT

```
Hasil missing value:
 target                    0
radius_mean               0
texture_mean              0
perimeter_mean            0
area_mean                 0
smoothness_mean           0
compactness_mean          0
concavity_mean            0
concave points_mean       0
symmetry_mean             0
fractal_dimension_mean    0
radius_se                 0
texture_se                0
perimeter_se              0
area_se                   0
smoothness_se             0
compactness_se            0
concavity_se              0
concave points_se         0
symmetry_se               0
fractal_dimension_se      0
radius_worst              0
texture_worst             0
perimeter_worst           0
area_worst                0
smoothness_worst          0
compactness_worst         0
concavity_worst           0
concave points_worst      0
symmetry_worst            0
fractal_dimension_worst   0
dtype: int64
```

Gambar 3. Pemeriksaan data duplikat

Kita akan melakukan pengecekan rows yang duplikat.

IN [72]:

```
# Memeriksa duplikat data
data.duplicated().sum()
```

OUTPUT

```
np.int64(0)
```

Pemeriksaan outlier dilakukan menggunakan metode Interquartile Range (IQR), yaitu dengan melihat nilai yang berada di luar rentang Q1 − 1,5×IQR hingga Q3 + 1,5×IQR. Pada hasil pemeriksaan, beberapa fitur memiliki outlier. Setelah fungsi penghapusan outlier dijalankan sesuai notebook, ukuran data berubah dari 569 menjadi 398 baris dengan 31 kolom.

Gambar 4. Deteksi outlier menggunakan IQR

IN [77]:

```
# Mencari tahu data yang outliers dengan mendefinisikan fungsi
continous_features = list(numerical_col.columns)
print("Fitur kontinu:", continous_features)

def outliers(data_out, drop = False):
    for each_feature in data_out.columns:
        feature_data = data_out[each_feature]
        Q1 = np.percentile(feature_data, 25.)
        Q3 = np.percentile(feature_data, 75.)
        IQR = Q3-Q1
        outlier_step = IQR * 1.5
        outliers = feature_data[~((feature_data >= Q1 - outlier_step) & (feature_data
<= Q3 + outlier_step))].index.tolist()
        if not drop:
            print('For the feature {}, Num of Outliers is {}'.format(each_feature,
len(outliers)))
        if drop:
            data.drop(outliers, inplace = True, errors = 'ignore')
            print('Outliers from {} feature removed'.format(each_feature))

# Menampilkan data yang outlier
outliers(data[continous_features])
```

OUTPUT

```
Fitur kontinu: ['radius_mean', 'texture_mean', 'perimeter_mean', 'area_mean',
'smoothness_mean', 'compactness_mean', 'concavity_mean', 'concave points_mean',
'symmetry_mean', 'fractal_dimension_mean', 'radius_se', 'texture_se', 'perimeter_se',
'area_se', 'smoothness_se', 'compactness_se', 'concavity_se', 'concave points_se',
'symmetry_se', 'fractal_dimension_se', 'radius_worst', 'texture_worst',
'perimeter_worst', 'area_worst', 'smoothness_worst', 'compactness_worst',
'concavity_worst', 'concave points_worst', 'symmetry_worst', 'fractal_dimension_worst']
For the feature radius_mean, Num of Outliers is 14
For the feature texture_mean, Num of Outliers is 7
For the feature perimeter_mean, Num of Outliers is 13
For the feature area_mean, Num of Outliers is 25
For the feature smoothness_mean, Num of Outliers is 6
For the feature compactness_mean, Num of Outliers is 16
For the feature concavity_mean, Num of Outliers is 18
For the feature concave points_mean, Num of Outliers is 10
For the feature symmetry_mean, Num of Outliers is 15
```

Gambar 5. Hasil penghapusan outlier

IN [78]:

```
# Menghapus outlier dengan fungsi sebelumnya
outliers(data[continous_features], drop=True)
print("Ukuran data setelah outlier dihapus:", data.shape)
```

OUTPUT

```
Outliers from radius_mean feature removed
Outliers from texture_mean feature removed
Outliers from perimeter_mean feature removed
Outliers from area_mean feature removed
Outliers from smoothness_mean feature removed
Outliers from compactness_mean feature removed
Outliers from concavity_mean feature removed
Outliers from concave points_mean feature removed
Outliers from symmetry_mean feature removed
Outliers from fractal_dimension_mean feature removed
Outliers from radius_se feature removed
Outliers from texture_se feature removed
Outliers from perimeter_se feature removed
Outliers from area_se feature removed
Outliers from smoothness_se feature removed
Outliers from compactness_se feature removed
Outliers from concavity_se feature removed
Outliers from concave points_se feature removed
Outliers from symmetry_se feature removed
Outliers from fractal_dimension_se feature removed
Outliers from radius_worst feature removed
Outliers from texture_worst feature removed
Outliers from perimeter_worst feature removed
Outliers from area_worst feature removed
Outliers from smoothness_worst feature removed
Outliers from compactness_worst feature removed
Outliers from concavity_worst feature removed
Outliers from concave points_worst feature removed
Outliers from symmetry_worst feature removed
Outliers from fractal_dimension_worst feature removed
Ukuran data setelah outlier dihapus: (398, 31)
```

Catatan: pada notebook, bagian INVALID_VALUES dibiarkan sebagai dictionary kosong. Oleh karena itu, tidak ada nilai yang ditandai secara khusus sebagai nilai tidak valid pada langkah tersebut. Hasil pemeriksaan missing value tetap menunjukkan tidak ada nilai kosong.

### 2.3 Exploratory Data Analysis (EDA)

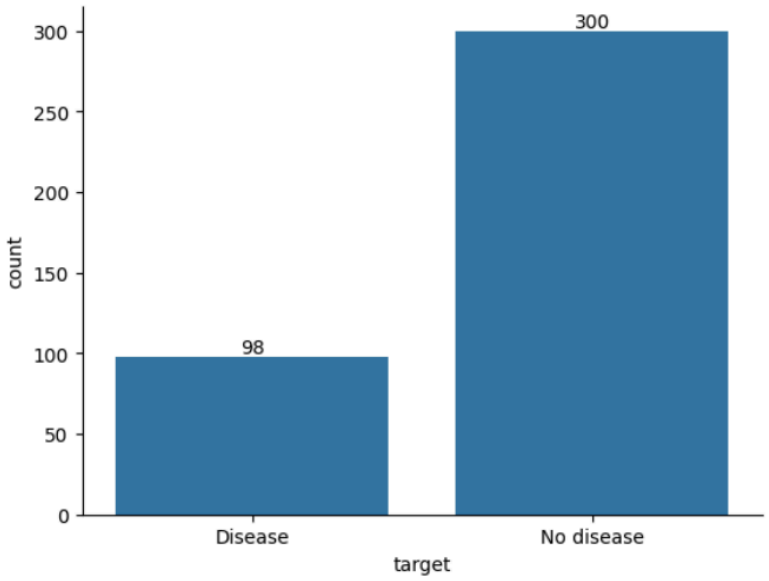
EDA dilakukan untuk memahami distribusi data, statistik deskriptif, dan proporsi kelas target. Setelah pembersihan outlier, distribusi target menunjukkan 75,4% data berada pada kelas No disease dan 24,6% pada kelas Disease. Perbedaan proporsi tersebut menunjukkan bahwa kelas target tidak seimbang (imbalanced).

Gambar 6. Distribusi kelas target

IN [82]:

```
# Memeriksa data imbalance
ax = sns.countplot(x='target', data=data)
for label in ax.containers:
    ax.bar_label(label)
sns.despine()
plt.show()
print(data['target'].value_counts(normalize=True).round(3))
```

OUTPUT



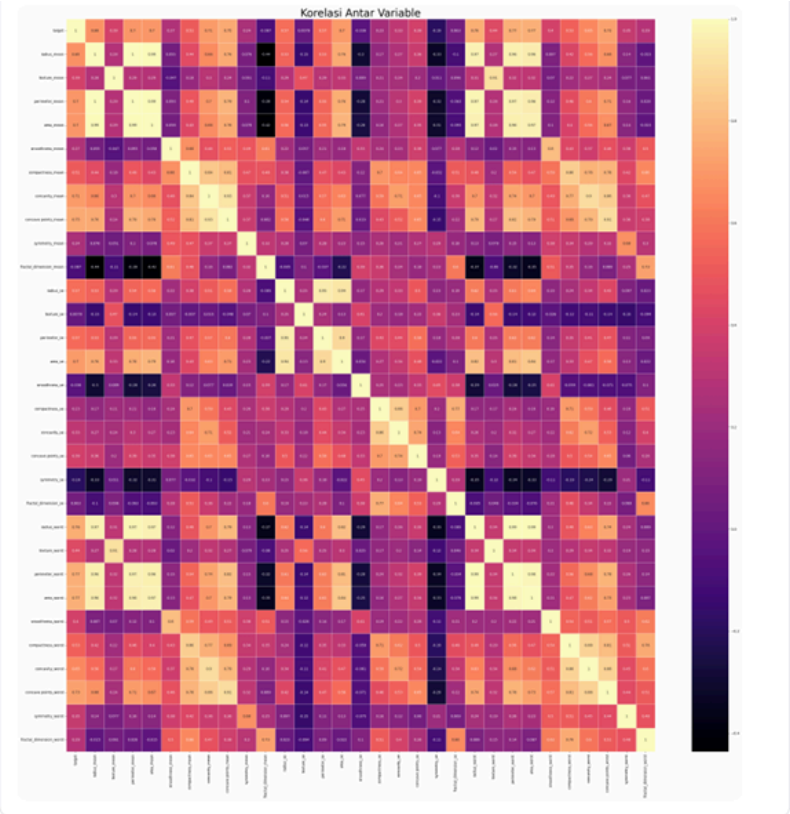

```
target
No disease    0.754
Disease       0.246
Name: proportion, dtype: float64
```

Statistik deskriptif juga digunakan untuk melihat nilai rata-rata, standar deviasi, nilai minimum, kuartil, median, dan maksimum dari fitur numerik. Selain itu, visualisasi histogram, boxplot, scatter plot, dan heatmap korelasi digunakan untuk membantu melihat pola dan hubungan antarfitur.

### 2.4 Feature Selection

Feature selection dilakukan dengan menghitung korelasi Pearson antara setiap fitur dan target. Fitur kemudian diurutkan berdasarkan nilai korelasi absolut. Sembilan fitur dengan korelasi absolut tertinggi dipilih untuk digunakan pada tahap pemodelan, yaitu area_worst, perimeter_worst, radius_worst, concave points_mean, concave points_worst, concavity_mean, area_se, area_mean, dan perimeter_mean.

Gambar 7. Heatmap korelasi




Gambar 8. Sembilan fitur terpilih

```
IN [92]:

# Memilih fitur dengan korelasi absolut tertinggi terhadap target (ubah K bila perlu)
K = min(9, df.shape[1]-1)
korelasi_target =
cor_matrix['target'].drop('target').abs().sort_values(ascending=False)
variabel = korelasi_target.head(K).index.tolist()
print("Fitur terpilih:", variabel)
analyze = df[variabel]

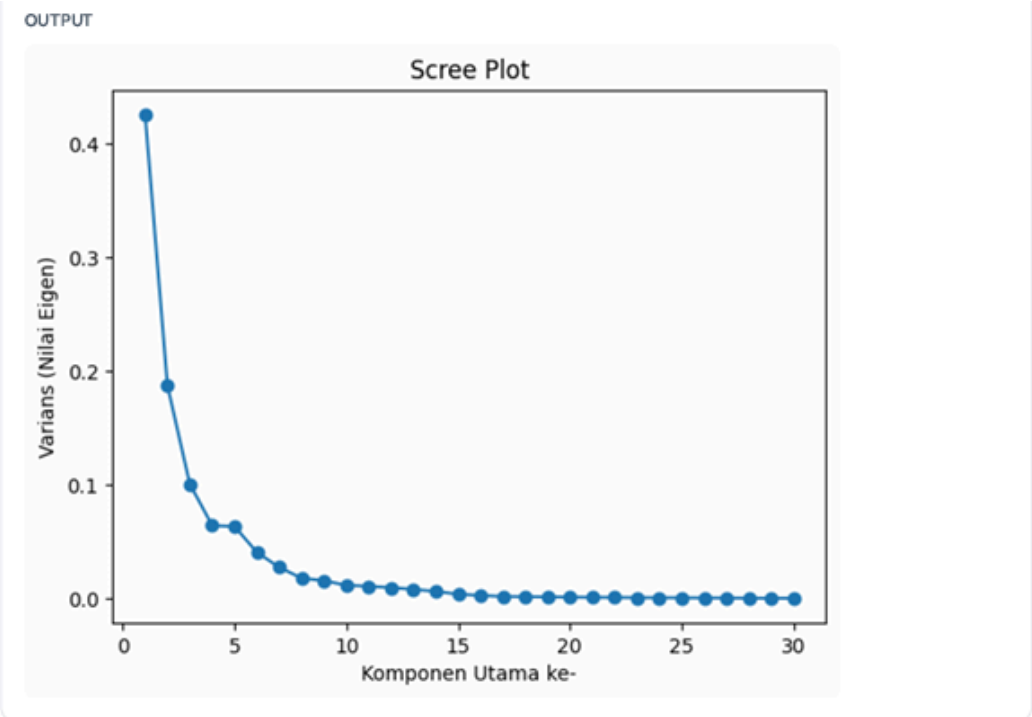
OUTPUT

Fitur terpilih: ['area_worst', 'perimeter_worst', 'radius_worst', 'concave
points_mean', 'concave points_worst', 'concavity_mean', 'area_se', 'area_mean',
'perimeter_mean']
```

### 2.5 Scaling dan Dimensionality Reduction

Sebelum PCA, fitur numerik distandarisasi menggunakan StandardScaler agar skala antarvariabel lebih sebanding. Selanjutnya, Principal Component Analysis (PCA) diterapkan untuk mereduksi dimensi data. Notebook menggunakan hingga sembilan komponen utama dan menampilkan scree plot serta fitur-fitur yang memiliki kontribusi besar pada masing-masing komponen.

Gambar 9. Scatter plot PCA




Gambar 10. Fitur dominan pada komponen utama

IN [97]:

```
feature_names = df.drop('target', axis=1).columns.to_list()
component_names = [f"PC{i+1}" for i in range(pca.n_components_)]

for component, component_name in zip(pca.components_, component_names):
    feature_indices = component.argsort()[::-1]
    retained_features = [feature_names[idx] for idx in
feature_indices[:pca.n_components_]]
    print(f"{component_name}: {retained_features}")
```

OUTPUT

```
PC1: ['concave points_mean', 'concavity_mean', 'concave points_worst',
'perimeter_worst', 'concavity_worst', 'radius_worst', 'area_worst', 'perimeter_mean',
'compactness_mean']
PC2: ['fractal_dimension_mean', 'fractal_dimension_se', 'fractal_dimension_worst',
'smoothness_mean', 'smoothness_se', 'compactness_se', 'smoothness_worst',
'compactness_mean', 'concavity_se']
PC3: ['texture_se', 'symmetry_se', 'radius_se', 'perimeter_se', 'smoothness_se',
'texture_mean', 'area_se', 'texture_worst', 'concave points_se']
PC4: ['symmetry_mean', 'smoothness_mean', 'symmetry_worst', 'smoothness_worst',
'symmetry_se', 'radius_se', 'smoothness_se', 'area_se', 'perimeter_se']
PC5: ['texture_worst', 'texture_mean', 'texture_se', 'symmetry_worst',
'smoothness_worst', 'fractal_dimension_worst', 'compactness_worst', 'concavity_worst',
'symmetry_mean']
PC6: ['symmetry_worst', 'symmetry_mean', 'symmetry_se', 'compactness_se',
'compactness_worst', 'concavity_se', 'concavity_worst', 'texture_mean',
'fractal_dimension_se']
PC7: ['fractal_dimension_worst', 'radius_se', 'perimeter_se', 'fractal_dimension_se',
'fractal_dimension_mean', 'area_se', 'compactness_worst', 'area_worst', 'radius_worst']
PC8: ['smoothness_se', 'smoothness_worst', 'symmetry_worst', 'symmetry_se',
'concavity_worst', 'fractal_dimension_worst', 'area_worst', 'area_se',
'fractal_dimension_se']
PC9: ['symmetry_se', 'fractal_dimension_se', 'area_mean', 'radius_mean',
'perimeter_mean', 'fractal_dimension_mean', 'compactness_mean',
'fractal_dimension_worst', 'area_worst']
```

*Principal Component Analysis* (PCA) dilakukan setelah tahap *feature selection* untuk menangani sisa interaksi multikolinearitas kompleks serta mereduksi dimensi data tanpa menghilangkan varians informasi penting. Meskipun 9 fitur teratas telah dipilih berdasarkan nilai korelasi tertinggi terhadap target, beberapa variabel fisik (seperti area_worst, perimeter_worst, dan radius_worst) masih memiliki keterikatan korelasi yang sangat tinggi satu sama lain (*multicollinearity*). PCA di sini bertindak sebagai langkah penyempurnaan (*fine-tuning*) untuk mentransformasikan 9 fitur terkorelasi tersebut menjadi 9 Komponen Utama (PC1–PC9) yang bersifat saling bebas (*orthogonal*), sehingga memunculkan kombinasi linier baru yang murni dari aspek varians tanpa adanya redundansi informasi.

Dampak utama dari penerapan PCA ini mencakup stabilitas matematis model, efisiensi komputasi, serta peningkatan batas pemisahan kelas (*decision boundary*). Bagi model berbasis linier seperti *Logistic Regression*, penghilangan multikolinearitas melalui variabel *ortogonal* mencegah pembobotan

koefisien menjadi tidak stabil (*over-sensitivity*), sehingga estimasi probabilitas prediksi menjadi lebih presisi dan konsisten. Selain itu, grafik *Scatterplot* menunjukkan bahwa komponen-komponen utama awal (seperti PC1 dan PC2) mampu menangkap proporsi akumulasi varians data paling dominan. Dampaknya, representasi data menjadi jauh lebih ringkas, mengurangi beban komputasi pada algoritma pembelajaran, serta membantu model membedakan kelas penyakit secara lebih fokus dan stabil.

Hasil Capstone I berupa dataset yang telah dibersihkan dan disimpan sebagai data.csv. Dataset ini menjadi masukan untuk tahap pemodelan pada Capstone II.

## BAB III. CAPSTONE II – MODELING

### 3.1 Persiapan Data dan Pemisahan Dataset

Pada tahap ini, dataset hasil Capstone I dimuat kembali. Target dipastikan dalam bentuk angka 0 dan 1. Berdasarkan korelasi absolut terhadap target, sembilan fitur terpilih digunakan sebagai variabel independen (X), sedangkan target menjadi variabel dependen (y). Data kemudian distandarisasi menggunakan StandardScaler dan dibagi menjadi data latih serta data uji dengan perbandingan 80:20 dan random_state 42.

Hasil pembagian menghasilkan 318 data latih dengan 9 fitur dan 80 data uji dengan 9 fitur. Jumlah label pada data latih adalah 318 dan pada data uji adalah 80.

Gambar 11. Pembagian data latih dan data uji

IN [7]:

```
# Mendefinisikan pemisahan data (80:20, random state 42)
X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.2,
random_state=42)

# Menampilkan hasil pemisahan berdasarkan kolom dan baris
print(X_train.shape)
print(X_test.shape)
print(y_train.shape)
print(y_test.shape)
```

OUTPUT

```
(318, 9)
(80, 9)
(318,)
(80,)
```

### 3.2 Pembangunan Model Klasifikasi

Empat algoritma klasifikasi digunakan untuk membandingkan performa model, yaitu Logistic Regression, Decision Tree, Random Forest, dan Multi-Layer Perceptron (MLP). Evaluasi awal dilakukan menggunakan accuracy, precision, recall, dan F1-score.

| Model | Accuracy | Precision kelas 1 | Recall kelas 1 | F1-score kelas 1 |
|---|---|---|---|---|
| **Logistic Regression** | 0,9000 | 0,87 | 0,80 | 0,83 |
| **Decision Tree** | 0,9000 | 0,90 | 0,76 | 0,83 |
| **Random Forest** | 0,8875 | 0,86 | 0,76 | 0,81 |
| **MLP** | 0,9250 | 0,95 | 0,80 | 0,87 |

Berdasarkan evaluasi awal, MLP memperoleh akurasi tertinggi sebesar 92,5%, disusul Logistic Regression dan Decision Tree masing-masing 90%. Random Forest memperoleh akurasi 88,75%. Pada kelas positif (ganas), MLP juga menghasilkan precision dan F1-score tertinggi, sedangkan recall kelas positif tertinggi pada hasil awal adalah Logistic Regression sebesar 0,80 dan MLP sebesar 0,80.

Gambar 12. Hasil evaluasi Logistic Regression

```
IN [9]:

  clf = LogisticRegression(max_iter=1000)
  # train the classifier
  clf.fit(X_train, y_train)
  y_pred = clf.predict(X_test)
  # printing the test accuracy
  print("The test accuracy score of Logistic Regression Classifier is ",
  accuracy_score(y_test, y_pred))

OUTPUT

  The test accuracy score of Logistic Regression Classifier is  0.9
```

```
IN [10]:

  print(classification_report(y_test, y_pred))

OUTPUT

              precision    recall  f1-score   support

           0       0.91      0.95      0.93        55
           1       0.87      0.80      0.83        25

    accuracy                           0.90        80
   macro avg       0.89      0.87      0.88        80
weighted avg       0.90      0.90      0.90        80
```

Gambar 13. Hasil evaluasi MLP

```
IN [15]:

  clf = MLPClassifier(max_iter=1000, random_state=42)
  # train the classifier
  clf.fit(X_train, y_train)
  y_pred = clf.predict(X_test)
  # printing the test accuracy
  print("The test accuracy score of MLP Classifier is ", accuracy_score(y_test, y_pred))

OUTPUT

  The test accuracy score of MLP Classifier is  0.925
```

```
IN [16]:

  print(classification_report(y_test, y_pred))

OUTPUT

              precision    recall  f1-score   support

           0       0.92      0.98      0.95        55
           1       0.95      0.80      0.87        25

    accuracy                           0.93        80
   macro avg       0.93      0.89      0.91        80
weighted avg       0.93      0.93      0.92        80
```

### 3.3 Hyperparameter Tuning

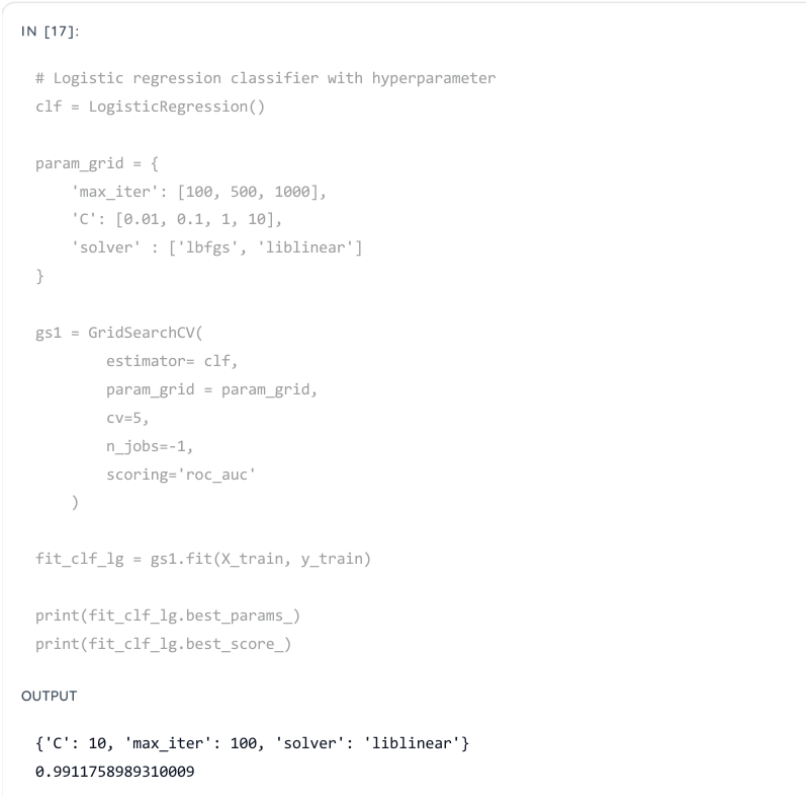
Hyperparameter tuning dilakukan menggunakan GridSearchCV dengan validasi silang 5-fold dan scoring ROC-AUC. Tujuannya adalah mencari kombinasi parameter yang memberikan performa terbaik pada data latih. Hasil parameter terbaik yang tercatat adalah sebagai berikut:

| Model | Parameter terbaik | Best CV ROC-AUC |
|---|---|---|
| **Logistic Regression** | C = 10; max_iter = 100; solver = liblinear | 0,9912 |
| **Random Forest** | Criterion = entropy; max_depth = 5; n_estimators = 100 | 0,9879 |

| | | |
|---|---|---|
| **Decision Tree** | Criterion = gini; max_depth = 3; min_samples_leaf = 5 | 0,9684 |
| **MLP** | Activation = tanh; hidden_layer_sizes = (50,50); solver = adam | 0,9902 |

Setelah tuning, akurasi Logistic Regression meningkat dari 90% menjadi 93,75% pada laporan pipeline akhir, dengan recall kelas positif meningkat dari 0,80 menjadi 0,84 pada evaluasi hasil tuning. Hasil Random Forest setelah tuning menunjukkan akurasi 91%, Decision Tree sekitar 89%, dan MLP sekitar 93%.

Gambar 14. Proses tuning Logistic Regression



```
IN [17]:

  # Logistic regression classifier with hyperparameter
  clf = LogisticRegression()

  param_grid = {
      'max_iter': [100, 500, 1000],
      'C': [0.01, 0.1, 1, 10],
      'solver' : ['lbfgs', 'liblinear']
  }

  gs1 = GridSearchCV(
          estimator= clf,
          param_grid = param_grid,
          cv=5,
          n_jobs=-1,
          scoring='roc_auc'
      )

  fit_clf_lg = gs1.fit(X_train, y_train)

  print(fit_clf_lg.best_params_)
  print(fit_clf_lg.best_score_)

OUTPUT

  {'C': 10, 'max_iter': 100, 'solver': 'liblinear'}
  0.9911758989310009
```

### 3.4 Evaluasi ROC-AUC dan Pemilihan Model

Evaluasi ROC-AUC digunakan untuk melihat kemampuan model membedakan kelas 0 dan kelas 1 berdasarkan probabilitas prediksi. Nilai AUC-ROC yang tercatat adalah Logistic Regression 0,9644; Random Forest 0,9331; Decision Tree 0,9265; dan MLP 0,9636. Logistic Regression memiliki nilai AUC-ROC tertinggi, meskipun selisihnya dengan MLP sangat kecil.

Notebook juga menghitung threshold terbaik menggunakan Youden's Index. Threshold yang diperoleh adalah 0,1310 untuk Logistic Regression, 0,3137 untuk Random Forest, 0,25 untuk Decision Tree, dan

0,3500 untuk MLP. Threshold yang lebih rendah dapat meningkatkan sensitivitas, tetapi berpotensi meningkatkan false positive.

Gambar 15. Kurva ROC dari empat model

IN [26]:

```
#Membuat ROC Analisis
def plot_roc_curves(y_test, y_pred_logreg, y_pred_rf, y_pred_dt, y_pred_mlp):
    plt.figure(figsize=(8, 6))

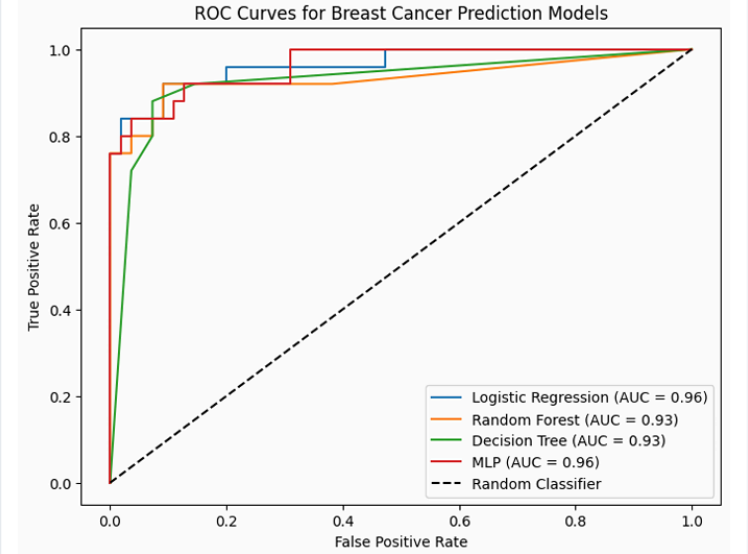
    # Calculate ROC curves for each model
    fpr_logreg, tpr_logreg, _ = roc_curve(y_test, y_pred_logreg)
    fpr_rf, tpr_rf, _ = roc_curve(y_test, y_pred_rf)
    fpr_dt, tpr_dt, _ = roc_curve(y_test, y_pred_dt)
    fpr_mlp, tpr_mlp, _ = roc_curve(y_test, y_pred_mlp)

    # Plot ROC curves
    plt.plot(fpr_logreg, tpr_logreg, label=f'Logistic Regression (AUC =
{auc_logreg:.2f})')
    plt.plot(fpr_rf, tpr_rf, label=f'Random Forest (AUC = {auc_rf:.2f})')
    plt.plot(fpr_dt, tpr_dt, label=f'Decision Tree (AUC = {auc_dt:.2f})')
    plt.plot(fpr_mlp, tpr_mlp, label=f'MLP (AUC = {auc_mlp:.2f})')

    # Plot random classifier
    plt.plot([0, 1], [0, 1], 'k--', label='Random Classifier')

    # Format the plot
    plt.xlabel('False Positive Rate')
    plt.ylabel('True Positive Rate')
    plt.title('ROC Curves for Breast Cancer Prediction Models')
    plt.legend()
    plt.show()

plot_roc_curves(y_test, y_pred_logreg, y_pred_rf, y_pred_dt, y_pred_mlp)
```




### 3.5 Penyimpanan Model Terbaik

Berdasarkan AUC-ROC, Logistic Regression dipilih sebagai model terbaik. Model disusun dalam pipeline yang menggabungkan StandardScaler dan estimator hasil tuning. Pipeline dilatih ulang dan disimpan menggunakan pickle dengan nama generate_breast_cancer.pkl. Output menunjukkan akurasi pipeline pada data uji sebesar 0,9375 atau 93,75%. Sembilan fitur yang digunakan dicatat agar urutan input pada aplikasi sama dengan urutan fitur saat pelatihan.

Gambar 16. Pemilihan dan penyimpanan model terbaik

IN [29]:

```
import pickle
from sklearn.pipeline import Pipeline
from sklearn.base import clone

# Memilih model terbaik berdasarkan AUC-ROC
hasil = {'Logistic Regression': (auc_logreg, fit_clf_lg),
         'Random Forest': (auc_rf, fit_clf_rf),
         'Decision Tree': (auc_dt, fit_clf_dt),
         'MLP': (auc_mlp, fit_clf_mlp)}
nama_terbaik = max(hasil, key=lambda k: hasil[k][0])
print("Model terbaik:", nama_terbaik, "dengan AUC =", hasil[nama_terbaik][0])

# Pipeline (scaler + model terbaik) dilatih ulang pada data asli, agar aplikasi
Streamlit
# bisa langsung menerima input mentah tanpa scaling manual
model_terbaik = Pipeline([('scaler', StandardScaler()),
                          ('model', clone(hasil[nama_terbaik][1].best_estimator_))])
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.2, random_state=42)
model_terbaik.fit(Xtr, ytr)
print("Akurasi pipeline pada data uji:", accuracy_score(yte,
model_terbaik.predict(Xte)))

# Menyimpan model terbaik dengan pickle
pklname = "generate_breast_cancer.pkl"

with open(pklname, 'wb') as file:
    pickle.dump(model_terbaik, file)
print("Fitur yang dipakai (urutan harus sama di aplikasi):", X.columns.tolist())
```

OUTPUT

```
Model terbaik: Logistic Regression dengan AUC = 0.9643636363636364
Akurasi pipeline pada data uji: 0.9375
Fitur yang dipakai (urutan harus sama di aplikasi): ['area_worst', 'perimeter_worst',
'radius_worst', 'concave points_mean', 'concave points_worst', 'concavity_mean',
'area_se', 'area_mean', 'perimeter_mean']
```

Perbandingan performansi model menunjukkan dinamika menarik antara kompleksitas arsitektur dan efisiensi model linear pada dataset berukuran kecil hingga sedang. Pada kondisi *baseline* (tanpa *tuning*), *Multi-Layer Perceptron* (MLP) mencatatkan akurasi awal tertinggi (92,5%) dibandingkan *Logistic Regression* (90,0%) karena struktur *neural network* secara alami lebih adaptif dalam memetakan batas keputusan non-linear. Namun, setelah dilakukan *hyperparameter tuning* dengan *GridSearchCV*, *Logistic Regression* yang dioptimalkan dengan parameter C=10 dan solver='liblinear' mengalami kenaikan performa signifikan, menjadi model terbaik dengan Akurasi 93,75%, skor AUC-ROC 0,9644, serta tingkat *Recall* kelas positif mencapai 84%. Peningkatan ini terjadi karena sub ruang 9 fitur utama terpilih memiliki batas pemisahan kelas yang relatif linear, sehingga model linear ter-regularisasi dapat menyesuaikan *decision boundary* secara sangat presisi. Sebaliknya, model MLP

dengan arsitektur dua *hidden layer* (50, 50) membutuhkan jumlah sampel data yang jauh lebih besar (*sample complexity*) agar proses *tuning* tidak tertahan pada kondisi *sub-optimal convergence*, menjadikan *Logistic Regression* sebagai model paling akurat, stabil, dan optimal untuk diimplementasikan.

## BAB IV. CAPSTONE III – DEPLOYMENT DASHBOARD

### 4.1 Persiapan Aplikasi

Capstone III mengembangkan dashboard prediksi menggunakan Streamlit. Aplikasi memuat model yang telah disimpan pada Capstone II melalui file generate_breast_cancer.pkl. File data.csv digunakan sebagai referensi rentang nilai untuk slider input. Modul yang digunakan antara lain os, streamlit, pandas, pickle, time, dan PIL.

### 4.2 Fungsi Input dan Prediksi

Pengguna dapat memasukkan data dengan dua cara, yaitu mengunggah file CSV atau mengisi fitur secara manual melalui slider. Jika pengguna mengunggah CSV, aplikasi memeriksa apakah seluruh kolom fitur yang diperlukan tersedia. Jika tidak, aplikasi menampilkan pesan kesalahan. Jika input manual digunakan, rentang slider ditentukan berdasarkan nilai minimum, maksimum, dan median dari data referensi.

Ketika tombol Predict! ditekan, aplikasi menjalankan model untuk menghasilkan prediksi kelas dan probabilitas kelas positif. Hasil ditampilkan sebagai Benign (Jinak) atau Malignant (Ganas), disertai probabilitas ganas.

Gambar 17. Kode fungsi dashboard dan prediksi

```
                lo, hi, med = float(ref[f].min()), float(ref[f].max()),
float(ref[f].median())
                data[f] = st.sidebar.slider(f, lo, hi, med, step=(hi - lo) / 200)
            features = pd.DataFrame(data, index=[0])
            return features

        input_df = user_input_features()

    if os.path.exists(IMAGE_PATH):
        img = Image.open(IMAGE_PATH)
        st.image(img, width=500)

    if st.sidebar.button('Predict!'):
        df = input_df
        st.write(df)
        prediction = loaded_model.predict(df)
        proba = loaded_model.predict_proba(df)[:, 1]
        st.subheader('Prediction: ')
        hasil = ['Benign (Jinak)' if p == 0 else 'Malignant (Ganas)' for p in
prediction]
        with st.spinner('Wait for it...'):
            time.sleep(2)
            for h, p in zip(hasil, proba):
                st.success(f"Prediction of this app is {h} (probabilitas ganas:
{p:.2%})")
```

OUTPUT

```
2026-11-04 09:37:17.964 No runtime found, using MemoryCacheStorageManager
```

Kita mendefinisikan fungsi breast_cancer() untuk membangun aplikasi prediksi kanker payudara. Model yang sudah di-training disimpan dalam file generate_breast_cancer.pkl berupa pipeline (scaling + model), sehingga input mentah langsung bisa diprediksi. Nama fitur dan urutannya dibaca langsung dari model, dan rentang slider dari data.csv. Hasil prediksi ditampilkan dengan st.success.

### 4.3 Menjalankan Aplikasi

Seluruh kode disimpan ke dalam file app.py. Aplikasi dijalankan melalui terminal menggunakan perintah streamlit run app.py. Fungsi main() memanggil dashboard berdasarkan pilihan kategori yang tersedia.

Gambar 18. Pemanggilan fungsi utama dan output notebook

IN [6]:

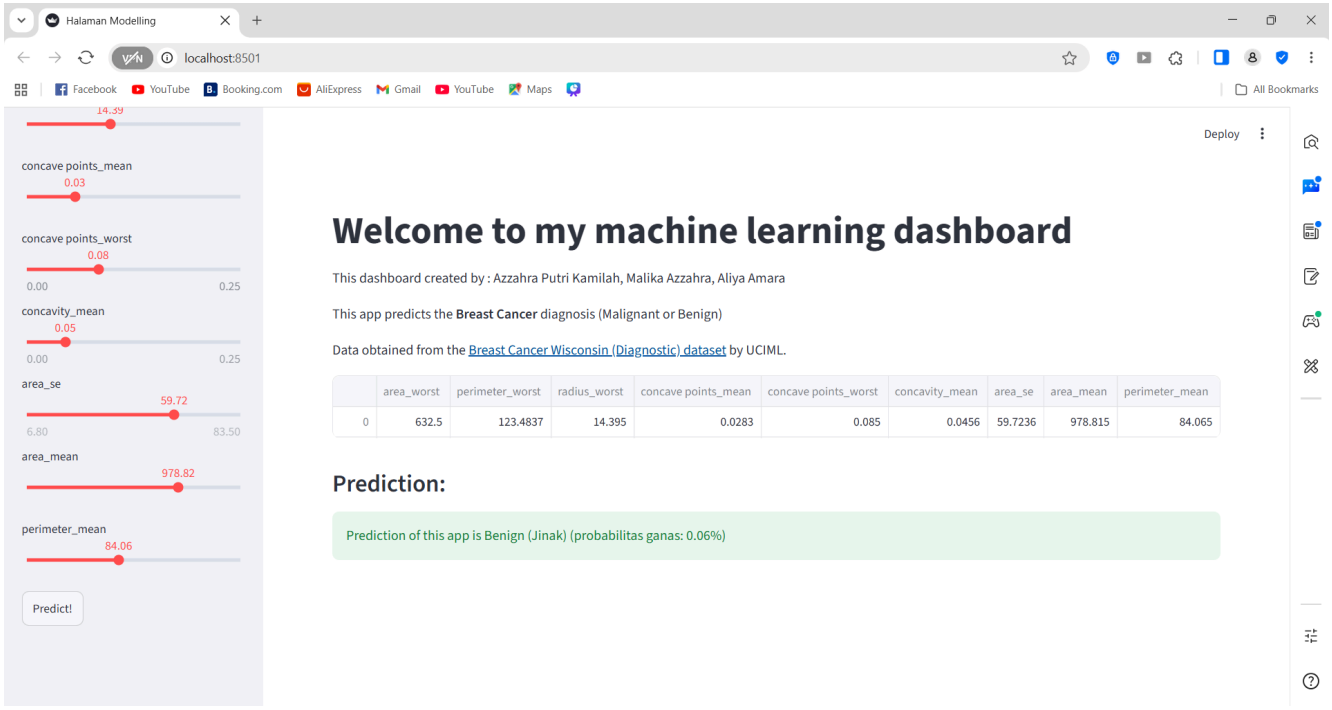
```
if __name__ == "__main__":
    main()
```

OUTPUT

```
2026-11-04 09:37:18.016 Thread 'MainThread': missing ScriptRunContext! This warning can
be ignored when running in bare mode.
2026-11-04 09:37:18.017 Thread 'MainThread': missing ScriptRunContext! This warning can
be ignored when running in bare mode.
2026-11-04 09:37:18.018 Thread 'MainThread': missing ScriptRunContext! This warning can
be ignored when running in bare mode.
2026-11-04 09:37:18.019 Thread 'MainThread': missing ScriptRunContext! This warning can
be ignored when running in bare mode.
2026-11-04 09:37:18.020 Thread 'MainThread': missing ScriptRunContext! This warning can
be ignored when running in bare mode.
2026-11-04 09:37:18.021 Thread 'MainThread': missing ScriptRunContext! This warning can
be ignored when running in bare mode.
2026-11-04 09:37:18.022 Thread 'MainThread': missing ScriptRunContext! This warning can
be ignored when running in bare mode.
2026-11-04 09:37:18.548 Thread 'Thread-3': missing ScriptRunContext! This warning can
be ignored when running in bare mode.
2026-11-04 09:37:18.552 Thread 'Thread-3': missing ScriptRunContext! This warning can
be ignored when running in bare mode.
2026-11-04 09:37:18.553 Thread 'Thread-3': missing ScriptRunContext! This warning can
be ignored when running in bare mode.
2026-11-04 09:37:22.116 Thread 'MainThread': missing ScriptRunContext! This warning can
be ignored when running in bare mode.
2026-11-04 09:37:22.118 Thread 'MainThread': missing ScriptRunContext! This warning can
be ignored when running in bare mode.
2026-11-04 09:37:22.119 Thread 'MainThread': missing ScriptRunContext! This warning can
be ignored when running in bare mode.
2026-11-04 09:37:22.120 No runtime found, using MemoryCacheStorageManager
2026-11-04 09:37:22.124 Thread 'MainThread': missing ScriptRunContext! This warning can
be ignored when running in bare mode.
2026-11-04 09:37:22.125 Thread 'MainThread': missing ScriptRunContext! This warning can
be ignored when running in bare mode.
2026-11-04 09:37:22.126 Thread 'MainThread': missing ScriptRunContext! This warning can
be ignored when running in bare mode.
2026-11-04 09:37:22.127 Thread 'MainThread': missing ScriptRunContext! This warning can
be ignored when running in bare mode.
2026-11-04 09:37:22.165 Thread 'MainThread': missing ScriptRunContext! This warning can
```

### 4.4 Hasil Dashboard

Gambar 19. Deploy Dashboard Breast Cancer




**Interpretasi :**

Dashboard berjalan benar dan memberi hasil Benign (Jinak) dengan probabilitas ganas hanya 0,06%. Artinya model sangat yakin sampel ini jinak. Keyakinan itu terutama ditentukan fitur concave points_mean (0,0283), concave points_worst (0,085), dan concavity_mean (0,0456), yang nilainya rendah. Fitur-fitur kecekungan sel itulah yang paling kuat membedakan tumor ganas dari jinak, dan semuanya termasuk dalam sembilan fitur terpilih. Tabel di tengah menampilkan satu baris input yang dikirim ke model, dan hasil prediksinya sesuai dengan nilai input tersebut.

## BAB V. KESIMPULAN

Berdasarkan tiga tahap capstone, proses dimulai dari persiapan dataset Breast Cancer Wisconsin (Diagnostic), pembersihan data, eksplorasi, pemilihan fitur, dan reduksi dimensi. Dataset awal memiliki 569 baris, kemudian setelah proses penghapusan outlier ukuran data menjadi 398 baris. Sembilan fitur dipilih berdasarkan korelasi absolut tertinggi terhadap target.

Pada tahap pemodelan, empat algoritma diuji. MLP memberikan akurasi awal tertinggi sebesar 92,5%, sedangkan Logistic Regression memperoleh AUC-ROC tertinggi sebesar 0,9644. Oleh karena pemilihan model akhir pada notebook didasarkan pada AUC-ROC, Logistic Regression dipilih dan disimpan sebagai pipeline dengan akurasi data uji 93,75%.

Pada tahap deployment, model diintegrasikan ke dalam aplikasi Streamlit yang menyediakan input manual maupun unggah CSV serta menampilkan hasil klasifikasi dan probabilitas ganas. Dengan

demikian, rangkaian capstone telah mencakup proses dari pengolahan data hingga rancangan aplikasi prediksi.